引言

在当今复杂的金融市场中，投资者面对着各种挑战和不确定性，因此，准确的市场分析和预测变得尤为重要。本文从宏观、中观和微观三个层次出发，探讨了如何通过Logistic回归模型、中观景气度指数的构建与应用，以及微观结构风险的度量，来更全面地分析市场。

一、宏观层面：Logistic回归对中证全指指数的预测能力分析在本次分析中，我们通过Logistic回归模型对中证全指指数未来走势的预测进行了深入研究。

结果显示，Logistic回归模型能够较好地捕捉市场的方向性变化，其预测结果与未来60个交易日内指数的涨跌方向高度相关。这意味着，模型在确定市场是否处于上升或下降趋势方面表现出色。

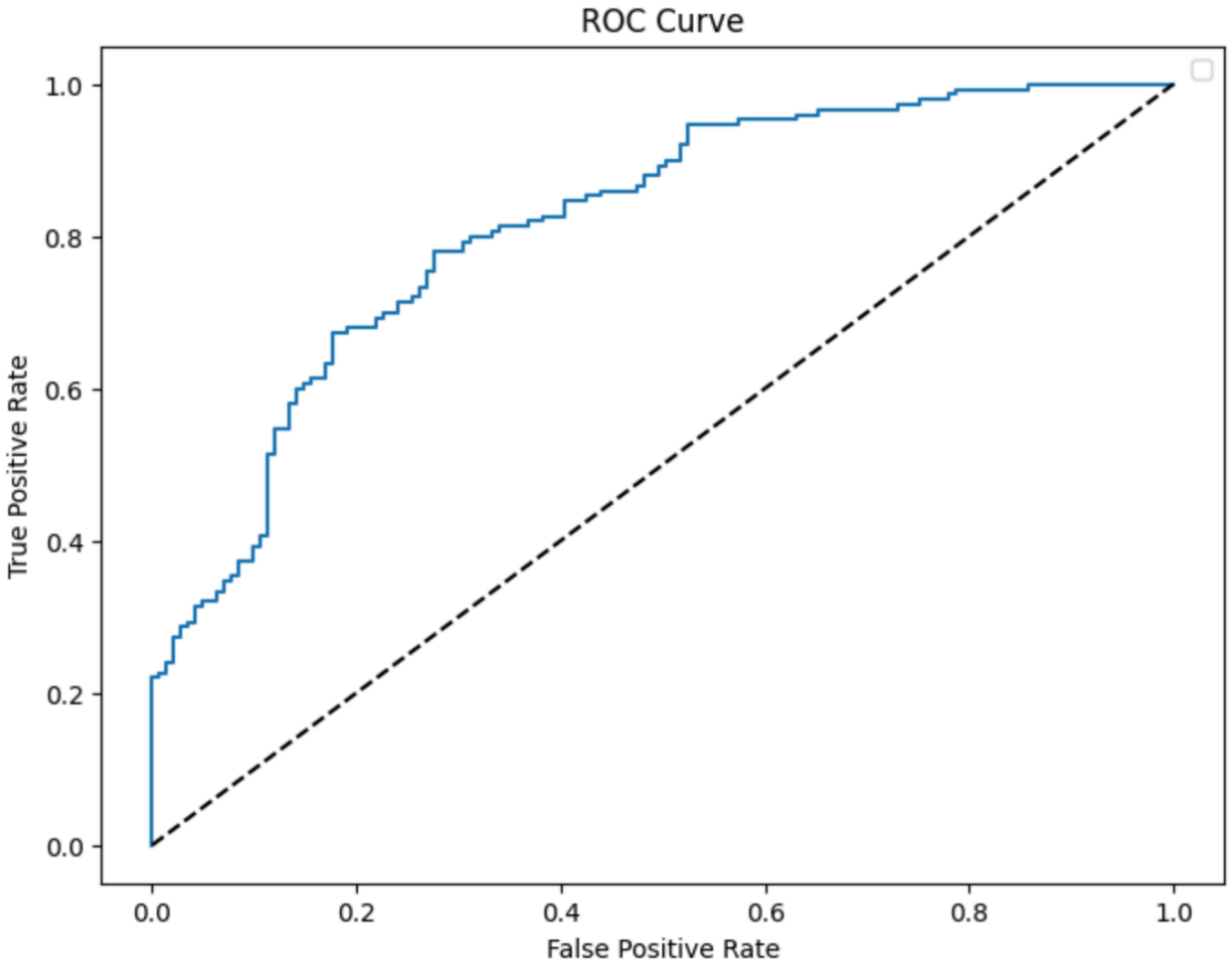
我们选取的宏观环境变量包括短期货币流动性、长端利率、汇率、信用状况、PMI等指标，并结合不同频率的数据，以确保模型的多维度和时效性。具体的变量如下：

- 短期货币流动性：银行间债券质押式回购（频率：月）
- 长端利率&汇率：3年期国债到期收益率（频率：日）、5/10年期国债到期收益率（频率：日）、人民币汇率指数（频率：日）
- 信用：社融新增人民币贷款（频率：月）、信用利差（全体产业债，频率：月）
- PMI：国内外制造业PMI数据，如中国的PMI新订单、PMI新出口订单（频率：月），以及美国、日本、欧元区的制造业PMI（频率：月）
- 其他：SHIBOR 3个月（频率：月）、R-007（频率：日）、2/10年期美债到期收益率（频率：日）、M1和M2同比变化（频率：月）

然而，值得注意的是，尽管模型在方向性判断上表现优异，但在预测市场趋势强度方面存在局限性。换句话说，Logistic回归模型在判断市场上升或下降趋势方面表现良好。然而，它在预测市场趋势的强度上还有一些局限性。

二、中观层面：中观景气度指数的构建与应用

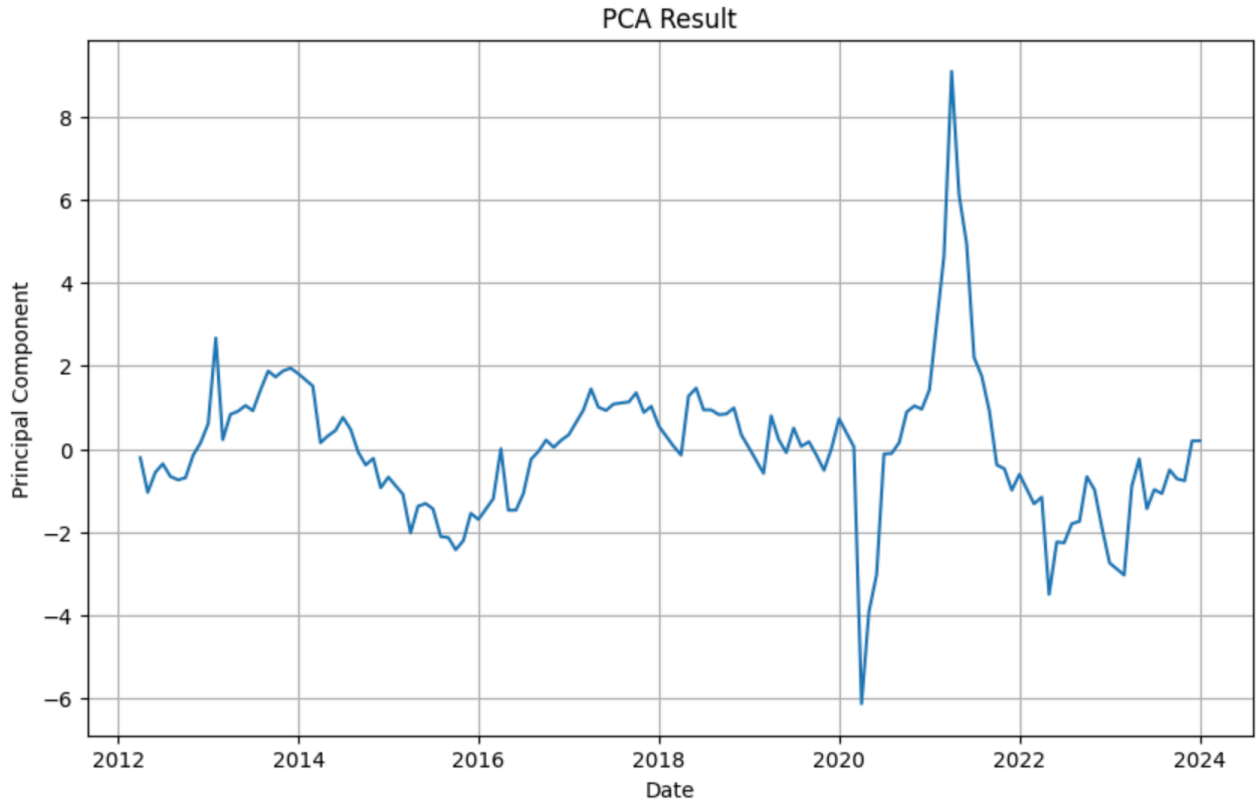
为了更全面地分析市场，我构建了一个中观景气度指数。中观景气度指数旨在通过宏观经济数据、企业盈利数据等多维度信息，衡量市场的内在景气程度。我们的构建步骤如下：



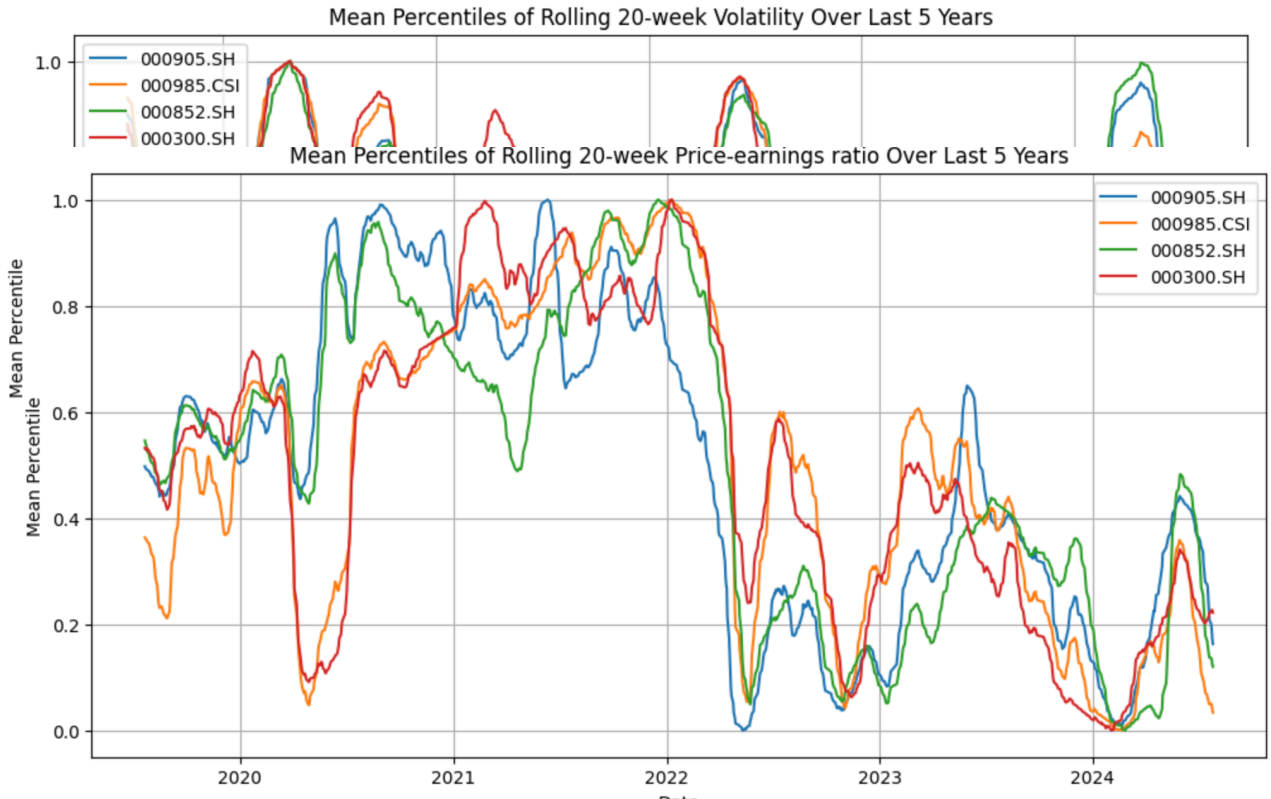
首先，我们收集并拼接了多种影响市场的因子数据，这些数据涵盖了经济的各个方面。接着，按照季度对数据进行采样，以确保数据的稳定性和可靠性。在此基础上，我们采用了主成分分析（PCA）技术，将这些多维因子合成为若干大类因子，以减少数据的维度并保留主要信息。这些大类因子随后用于回归模型，其目标是上证指数的归属母公司股东净利润同比增长率。为了适应市场的高频率变化，我们将这些因子的频率提升至日频，并用训练好的模型进行预测。具体来说，我们收集了多种市场影响因素的数据，并将这些数据简化为几个主要因素。然后，使用这些因素预测上证指数的盈利趋势。结果表明，该指数不仅与A股盈利趋势密切相关，而且在一定程度上具备市场领先预测能力，为投资决策提供了早期预警信号。

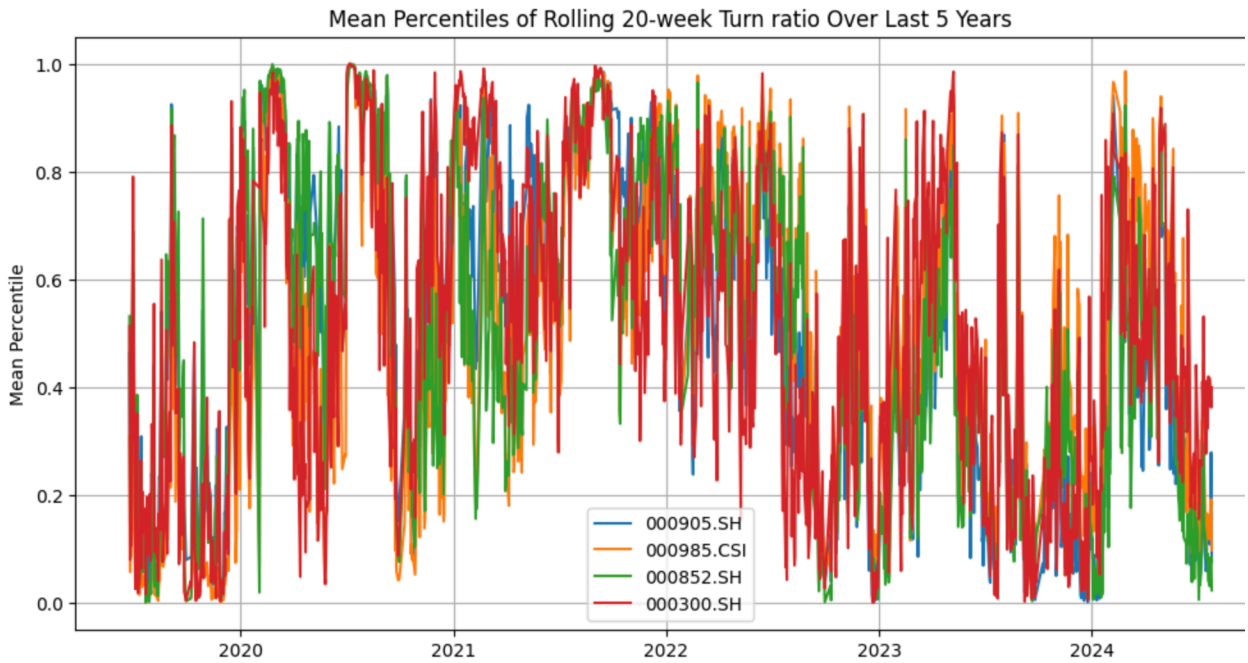
三、微观层面：微观结构风险度量与分析

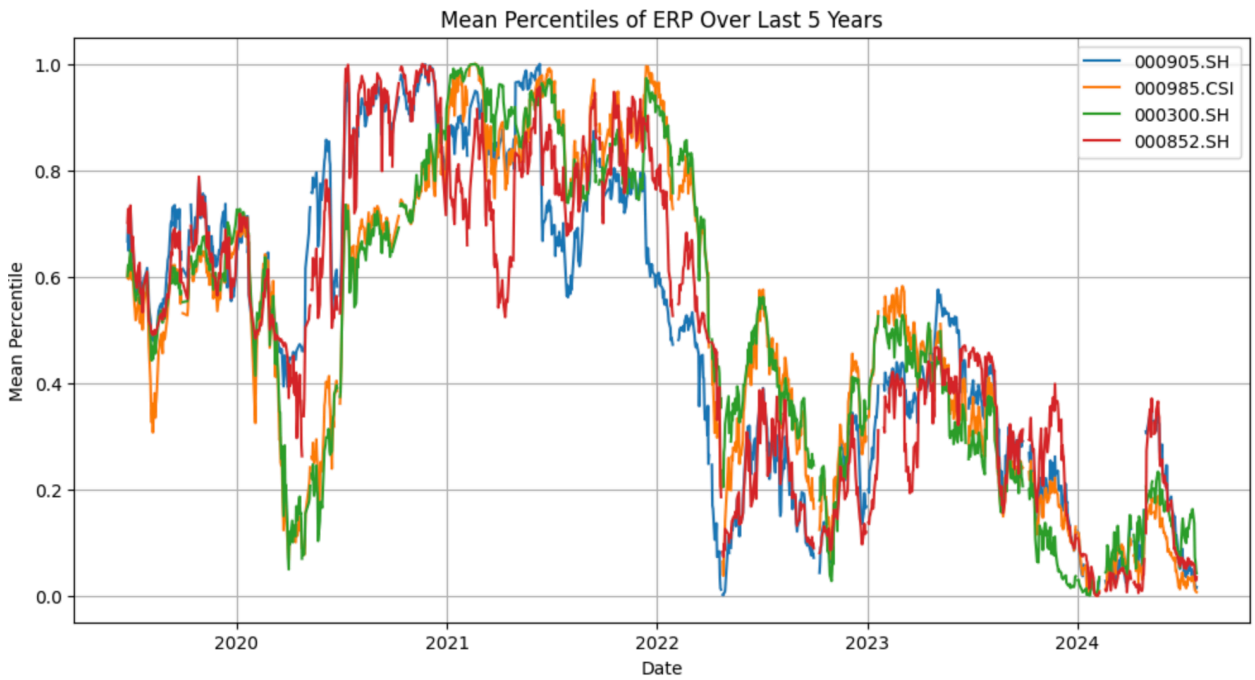
微观结构风险的度量在市场分析中同样至关重要。我们从估值水平、风险溢价和交易热度三个维度，对市场的微观结构风险进行了全面分析。首先，通过计算市盈率和市净率的历史分位数，评估当前市场的估值水平。其次，引入风险溢价的概念，通过反转历史数据来



衡量市场对风险资产的回报要求，获得市场的内在风险指标。此外，通过分析市场的交易热度，使用换手率分位数评估市场的流动性。最终，将估值、风险溢价和交易热度三个因子等权相加，合成了一个综合的微观结构风险指标，该指标表明市场的左侧风险特征，即市场在极端低估状态下通常伴随着较高的反转预期。




Mean Percentiles of Rolling 20-week Turn ratio Over Last 5 Years
Mean Percentile
1.0
0.8
0.6
0.4
0.2
0.0
2020
2021
2022
2023
2024
000905.SH
000985.CSI
000852.SH
000300.SH


Mean Percentiles of ERP Over Last 5 Years
Mean Percentile
1.0
0.8
0.6
0.4
0.2
0.0
2020
2021
2022
2023
2024
Date
000905.SH
000985.CSI
000300.SH
000852.SH

code: import pandas as pd

import numpy as np

```
from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score

import matplotlib.pyplot as plt

from sklearn.metrics import roc_curve

import matplotlib.pyplot as plt

df = pd.read_csv("/Users/haolinliu/Downloads/China_M1_YoY.csv")

df['date'] = pd.to_datetime(df['Name'], errors='coerce')

df = df.dropna(subset=['date'])

df.set_index('date', inplace=True)

df = df.fillna(method='bfill')

df = df.dropna(axis=1, how='all')
```

```
df = df.dropna(how='any')

results = {}

for column in df.columns:
    if column == 'Name':
        continue
    results[column] = pd.DataFrame()
    results[column][column] = df[column]
    results[column]['lag1'] = results[column][column].shift(1)
```

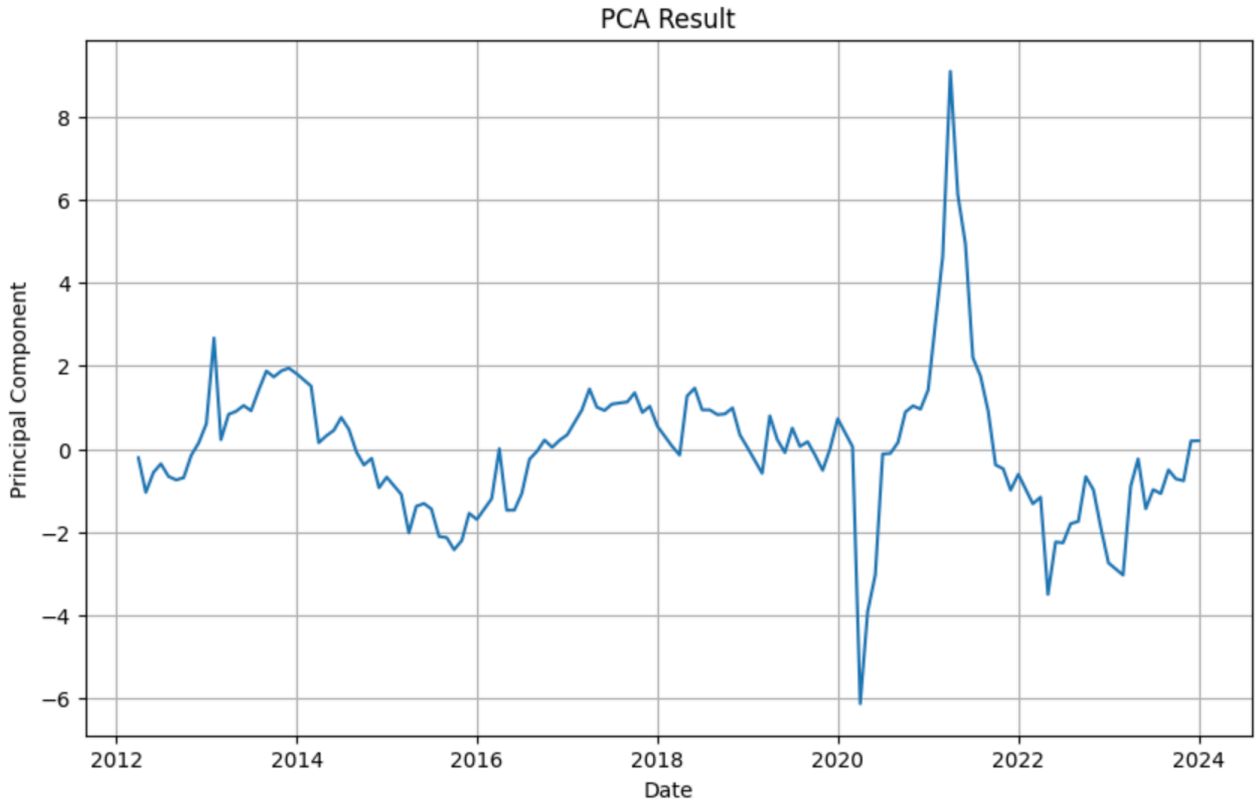
```
    results[column]['DIF'] = results[column][column] - results[column]['lag1']

    results[column]['EMA12'] = results[column][column].ewm(span=12,
adjust=False).mean()

    results[column]['EMA26'] = results[column][column].ewm(span=26,
adjust=False).mean()

    results[column]['DIF_EMA'] = results[column]['EMA12'] - results[column]['EMA26']



processed_df = pd.concat(results.values(), axis=1)



processed_df = processed_df.dropna()




data = pd.read_csv("/Users/haolinliu/Desktop/112233.csv", parse_dates=['交易日期'])

data = data.dropna(axis=1, how='all')
```

```
data.set_index('交易日期', inplace=True)



start_date = '2017-01-04'

end_date = '2023-03-27'

train_data = data.loc[start_date:end_date]

train_data['future_60d_return'] = train_data['收盘价'].shift(-60) / train_data['收盘价'] - 1

train_data['macro_environment'] = (train_data['future_60d_return'] > 0).astype(int)

train_data = train_data.dropna(subset=['future_60d_return'])

macro_environment_counts = train_data['macro_environment'].value_counts()



train_data = train_data.iloc[::-1]

train_data = train_data.join(processed_df, how='inner')

X = train_data[processed_df.columns]
```

```
y = train_data['macro_environment']


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


model = LogisticRegression()

model.fit(X_train_scaled, y_train)


y_pred = model.predict(X_test_scaled)

y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
```

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy}")

fpr, tpr, _ = roc_curve(y_test, y_pred_proba)

plt.figure(figsize=(8, 6))

plt.plot(fpr, tpr, label='Logistic Regression (AUC = {:.2f})'.format(roc_auc))

plt.plot([0, 1], [0, 1], 'k--')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.title('ROC Curve')

plt.legend()
```

```
plt.show()

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
from WindPy import w
w.start()
data =
w.edb("M0000548,S0027908,S0027703,S0027379,M5207101,M0000557,M0000559,M5567905
", "2001-12-31", "2023-12-31", "Fill=Previous")
df = data.Data


data.Fields = list(range(1, 9))
df=pd.DataFrame(data.Data,index=data.Fields,columns=data.Times)
df=df.T
df = df.dropna()
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
pca = PCA(n_components=1)
pca_data = pca.fit_transform(scaled_data)

pca_df = pd.DataFrame(data=pca_data, columns=['Principal Component'], index=df.index)
plt.figure(figsize=(10, 6))
plt.plot(pca_df.index, pca_df['Principal Component'])
plt.title('PCA Result')
plt.xlabel('Date')
plt.ylabel('Principal Component')
plt.grid()
plt.show()
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score, roc_curve
```




```
import matplotlib.pyplot as plt
from WindPy import w
w.start()
# 获取股票波动率数据
data = w.wsd("000905.SH,000985.CSI,000852.SH,000300.SH", "volatilityratio", "2019-06-25",
"2024-07-25", "CalculationTime=60;AnnualCoefficient=252;")
df = pd.DataFrame(data.Data, index=data.Codes, columns=data.Times).T

# 计算滚动平均值
df_rolling = df.rolling(window=20).mean()
df_rolling.dropna()


# 定义计算分位数的函数

def calculate_percentiles(df):
    df_percentiles = df.apply(lambda x: x.rank(pct=True))
    return df_percentiles
```

```
df_percentiles = calculate_percentiles(df_rolling)


# 绘制波动率分位数图
plt.figure(figsize=(12, 6))
plt.plot(df_percentiles.index, df_percentiles, label='Mean Percentile')

plt.title('Mean Percentiles of Rolling 20-week Volatility Over Last 5 Years')
plt.xlabel('Date')
plt.ylabel('Mean Percentile')
plt.legend(df_percentiles.columns)
plt.grid()
plt.show()

# 获取市盈率数据
pe_data= w.wsd("000905.SH,000985.CSI,000852.SH,000300.SH", "pe", "2019-06-25",
"2024-07-25", "ruleType=10;")
pe_df = pd.DataFrame(pe_data.Data, index=pe_data.Codes, columns=pe_data.Times).T
pe_df = pe_df.rolling(window=20).mean()

# 计算市盈率分位数
pe_df_percentiles = calculate_percentiles(pe_df)
plt.figure(figsize=(12, 6))
plt.plot(pe_df_percentiles.index, pe_df_percentiles, label='Mean Percentile')

plt.title('Mean Percentiles of Rolling 20-week Price-earnings ratio Over Last 5 Years')
plt.xlabel('Date')
plt.ylabel('Mean Percentile')
plt.legend(pe_df_percentiles.columns)
plt.grid()
plt.show()

# 获取换手率数据
turn_data = w.wsd("000905.SH,000985.CSI,000852.SH,000300.SH", "turn", "2019-06-25",
"2024-07-25", "")
turn_df = pd.DataFrame(turn_data.Data, index=turn_data.Codes, columns=turn_data.Times).T


turn_df_percentiles = calculate_percentiles(turn_df)
# 计算换手率分位数
plt.figure(figsize=(12, 6))
plt.plot(turn_df_percentiles.index, turn_df_percentiles, label='Mean Percentile')
```

```
plt.title('Mean Percentiles of Rolling 20-week Turn ratio Over Last 5 Years')
plt.xlabel('Date')
plt.ylabel('Mean Percentile')
plt.legend(turn_df_percentiles.columns)
plt.grid()
plt.show()
# 计算ERP分位数
data = w.wsd("000905.SH,000985.CSI,000300.SH,000852.SH", "pe", "2019-06-26",
"2024-07-26", "ruleType=10;")
df = pd.DataFrame(data.Data, index=data.Codes, columns=data.Times).T
df = 1/(df/100)
data_b = w.edb("S0059749", "2019-06-26", "2024-07-26", "Fill=Previous")
df_b = pd.DataFrame(data_b.Data, index=data_b.Codes, columns=data_b.Times).T
df_b = df_b.squeeze()

result = df.sub(df_b, axis=0)
result = -result
result = calculate_percentiles(result)
plt.figure(figsize=(12, 6))
plt.plot(result.index, result, label='Mean Percentile')

plt.title('Mean Percentiles of ERP Over Last 5 Years')
plt.xlabel('Date')
plt.ylabel('Mean Percentile')
plt.legend(result.columns)
plt.grid()
plt.show()
# 对齐所有分位数的DataFrame并计算它们的和
common_index =
df_percentiles.index.intersection(pe_df_percentiles.index).intersection(turn_df_percentiles.index
).intersection(result.index)
df_percentiles_aligned = df_percentiles.loc[common_index]
pe_df_percentiles_aligned = pe_df_percentiles.loc[common_index]
turn_df_percentiles_aligned = turn_df_percentiles.loc[common_index]
result_aligned = result.loc[common_index]

# 将所有分位数求和并计算平均值
summed_percentiles = df_percentiles_aligned.add(pe_df_percentiles_aligned, fill_value=0)
summed_percentiles = summed_percentiles.add(turn_df_percentiles_aligned, fill_value=0)
summed_percentiles = summed_percentiles.add(result_aligned, fill_value=0)
summed_percentiles = summed_percentiles / 4
```

```
# 绘制所有单独分位数和它们的总和
plt.figure(figsize=(12, 8))
plt.plot(df_percentiles_aligned.index, df_percentiles_aligned.mean(axis=1), label='Volatility
Percentile')
plt.plot(pe_df_percentiles_aligned.index, pe_df_percentiles_aligned.mean(axis=1), label='PE
Ratio Percentile')
plt.plot(turn_df_percentiles_aligned.index, turn_df_percentiles_aligned.mean(axis=1),
label='Turnover Percentile')
plt.plot(result_aligned.index, result_aligned.mean(axis=1), label='ERP Percentile')
plt.plot(summed_percentiles.index, summed_percentiles.mean(axis=1), label='Sum of
Percentiles', color='black', linewidth=2)

plt.title('Individual and Summed Percentiles of Different Market Indicators')
plt.xlabel('Date')
plt.ylabel('Percentile')
plt.legend()
plt.grid()
plt.show()
```